LINUX CASE STUDY

A Case Study Report on the Linux Operating System

CSC 266: Operating System

Submitted by:

bikash

Roll No.: 17

Section: A

Submitted to:

Department of Computer Science and Information Technology

Submission Date: _____

Table of Contents

1. Introduction 1
2. History 2
3. Kernel Modules 3
4. Process Management and Scheduling Approaches 4
5. Inter-process Communication 6
6. Memory Management Approaches 8
7. File System Management Approaches 9
8. Device Management Approaches 10
9. Summary 11
References 12

List of Tables

Table 1: Comparison of System V IPC and POSIX IPC 7

List of Figures

Figure 1: Layered architecture of the Linux operating system 1

Figure 2: Process state transition diagram 4

List of Abbreviations

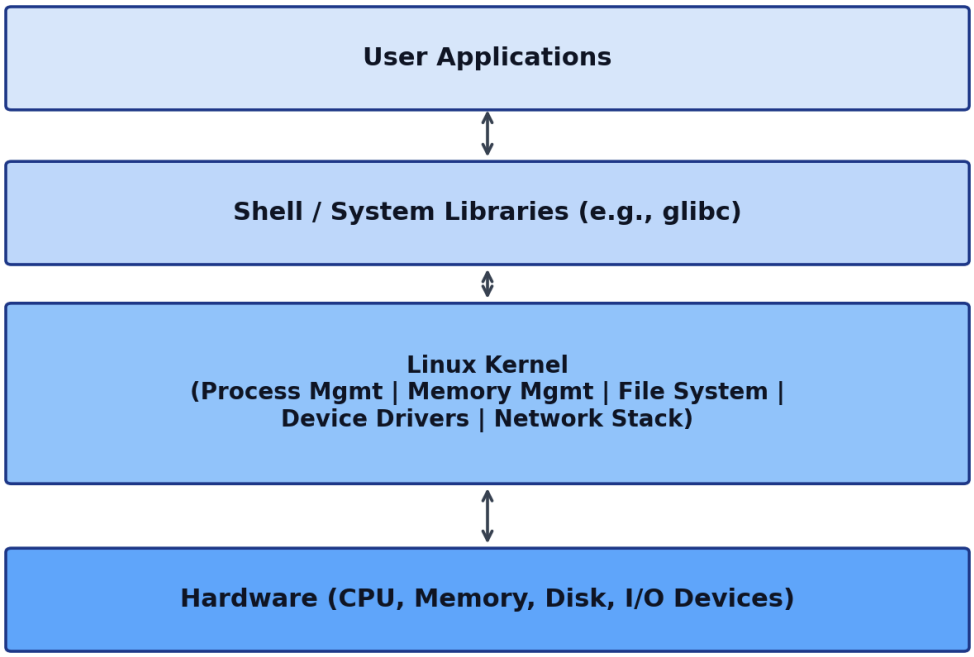
API	Application Programming Interface
BSD	Berkeley Software Distribution
CFS	Completely Fair Scheduler
CPU	Central Processing Unit
EEVDF	Earliest Eligible Virtual Deadline First
FAT	File Allocation Table
FIFO	First In, First Out
FS	File System
GNU	GNU's Not Unix
GPL	General Public License
I/O	Input/Output
IEEE	Institute of Electrical and Electronics Engineers
IPC	Inter-Process Communication
LKM	Loadable Kernel Module
LRU	Least Recently Used
LTS	Long Term Support
MMU	Memory Management Unit
OS	Operating System
PCB	Process Control Block
POSIX	Portable Operating System Interface
RAM	Random Access Memory
TLB	Translation Lookaside Buffer
UID	User Identifier
VFS	Virtual File System

1. Introduction

Linux is a free and open-source, Unix-like operating system kernel that today underpins a very large share of the world's computing infrastructure, spanning embedded devices, desktops, web servers, cloud platforms, mainframes, and supercomputers [1]. Strictly speaking, the term “Linux” refers only to the kernel itself, the core software component responsible for managing hardware resources and exposing system calls to user-space programs. A complete, installable operating system such as Ubuntu, Fedora, or Debian is more accurately described as a “GNU/Linux distribution”, since it combines the Linux kernel with the GNU project's compiler toolchain, shell, and core utilities, together with additional application software [3].

As of 2026, the Linux kernel is maintained collaboratively by thousands of contributors worldwide under the coordination of Linus Torvalds and the Linux Foundation. New mainline kernel releases appear roughly every nine to ten weeks, while designated Long Term Support (LTS) branches continue to receive security and bug-fix updates for several years after their initial release [6].

Architecturally, Linux follows a monolithic-but-modular design: the kernel executes as a single privileged process in kernel space, yet most of its functionality, including device drivers, file systems, and even entire subsystems, can be compiled as loadable kernel modules that are inserted into, or removed from, a running kernel on demand. This combines much of the raw performance of a monolithic kernel with the flexibility usually associated with a microkernel [1], [7]. Figure 1 illustrates the layered relationship between user applications, system libraries, the kernel, and the underlying hardware.



2. History

Linux was created in 1991 by Linus Torvalds, then a 21-year-old computer science student at the University of Helsinki, Finland [3], [5]. Torvalds began experimenting with low-level kernel programming in April 1991 on an Intel 80386-based personal computer, while running MINIX, a small Unix-like teaching operating system created by Andrew S. Tanenbaum [2], [3]. On 25 August 1991, Torvalds posted a now-famous message to the comp.os.minix newsgroup announcing his hobby project, deliberately downplaying its ambitions by stating that it would not be “big and professional like GNU” [5].

The first public source release, version 0.01, followed in September 1991, and a more usable version 0.02, comprising roughly ten thousand lines of code, was released in October 1991 [5]. Early Linux releases were distributed under a license that forbade commercial use; a pivotal change occurred with version 0.12 in 1992, when Torvalds re-licensed the kernel under the GNU General Public License (GPL), aligning it with the broader GNU project and enabling free redistribution and modification by anyone [5].

Through the 1990s and 2000s the kernel grew rapidly, moving from a handful of C source files to millions of lines of code, and adopted an even/odd version-numbering convention in which even minor versions denoted stable release series and odd minor versions denoted development branches [4]. The Open Source Development Labs, which Torvalds joined in 2003 to focus on kernel development, later merged with the Free Standards Group to form the Linux Foundation in January 2007, which continues to coordinate kernel development and maintain infrastructure to this day [4].

Today, Linux distributions power an estimated 96 percent of the world's top web servers, form the foundation of the Android mobile operating system, and run the overwhelming majority of the world's fastest supercomputers [5]. As of August 2026, the mainline kernel had progressed into the 7.x series, with the 6.18 and 6.12 branches maintained as long-term support releases [6], [4].

3. Kernel Modules

A Loadable Kernel Module (LKM), generally referred to simply as a “kernel module”, is a piece of object code that can be inserted into, or removed from, a running Linux kernel without requiring the system to be recompiled or rebooted [7]. Once loaded, a module becomes an integral part of the kernel, executing with full kernel privilege and direct access to kernel data structures. Device drivers, file system drivers, and network protocol handlers are among the most common uses of this mechanism [7], [8].

Kernel modules are compiled into object files with a .ko extension and can be loaded using the insmod utility, which issues an init_module system call that copies the module into kernel memory and immediately invokes the routine registered through the module's module_init() macro. This initialization routine typically registers the module's functions, such as a device driver's read and write operations, with the appropriate kernel subsystem [8]. Removing a module is performed with rmmod, which triggers the routine registered via module_exit() so the module can release any resources it acquired [8].

In practice, most Linux distributions rely on the higher-level modprobe command rather than raw insmod and rmmod calls, because modprobe automatically resolves and loads any modules that a requested module depends on, using a dependency map built by the depmod utility [7], [8]. The currently loaded modules on a running system can be inspected through the lsmod command or by reading the /proc/modules pseudo-file, and configuration options for individual modules are typically stored under /etc/modprobe.d/ [7], [8].

The loadable module mechanism gives Linux much of the extensibility associated with a microkernel design while retaining the performance characteristics of a monolithic kernel, since loaded modules run in the same address space as the rest of the kernel rather than communicating with it through message passing [1], [7].

4. Process Management and Scheduling Approaches

In Linux, as in most modern operating systems, a process is a program in execution, represented internally by a data structure (task_struct) that plays the role of a classical Process Control Block, recording the process's identifier, state, scheduling information, memory mappings, and open file descriptors [1], [2]. New processes are created almost exclusively through the fork() system call, which creates a near-exact copy of the calling process; the child process typically then calls one of the exec() family of system calls to replace its memory image with a new program. A parent process can wait for a child to terminate using wait() or waitpid(), and a process ends by calling exit() [1]. Linux also supports lightweight processes, or threads, created with the clone() system call, which allows a new task to share selected resources, such as the address space, with its parent, giving user-space threading libraries such as POSIX threads their underlying implementation [2].

During its lifetime, a process moves between a small number of states, conventionally described as new, ready, running, waiting (blocked), and terminated, as shown in Figure 2 [1], [2]. The kernel's scheduler is responsible for deciding, at each scheduling decision point, which ready process should be given the CPU.

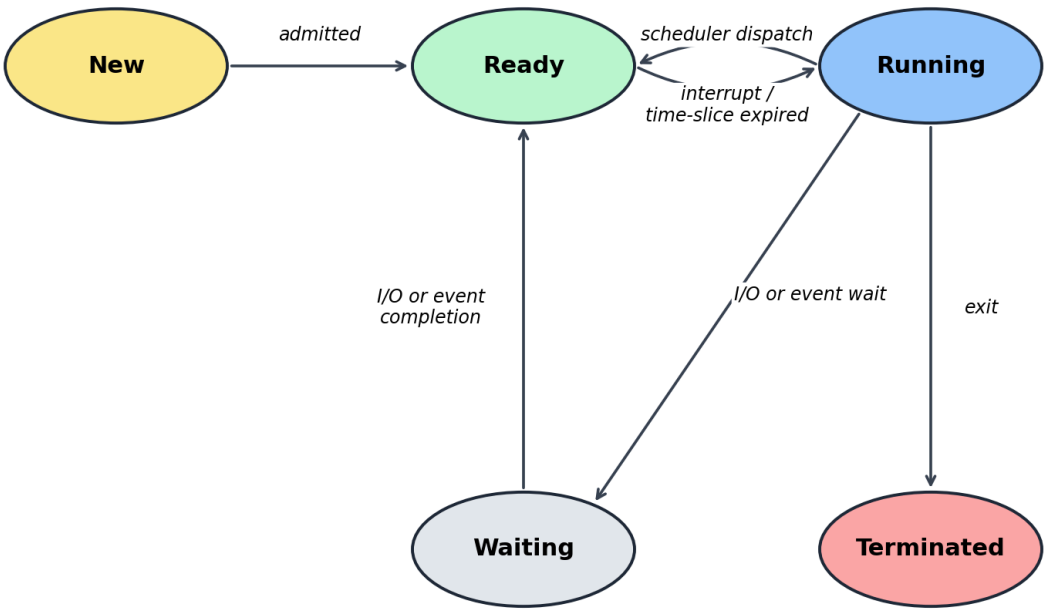


Figure 2: Process state transition diagram

Evolution of the CPU Scheduler

Linux distinguishes between real-time scheduling policies, such as SCHED_FIFO and

Although CFS served as Linux's default time-sharing scheduler for roughly sixteen years, its fairness guarantees depended on numerous tunable parameters and heuristics that had accumulated over time to handle interactive workloads, batch workloads, and everything in between, making the scheduler difficult to reason about in some edge cases [12]. Starting with kernel version 6.6, released in 2023, CFS was replaced by the Earliest Eligible Virtual Deadline First (EEVDF) scheduler, an algorithm originally proposed in a 1995 research paper by Ion Stoica and Hussein Abdel-Wahab and adapted for Linux by kernel developer Peter Zijlstra [11], [12]. Rather than relying purely on accumulated virtual runtime, EEVDF assigns each task a virtual eligible time and a virtual deadline, together with a “lag” value that expresses how far a task's received CPU time has diverged from its fair share, allowing the scheduler to make more predictable latency-versus-fairness trade-offs than CFS while reducing dependence on ad hoc heuristics [11], [12].

5. Inter-process Communication

Because processes in Linux normally execute in isolated address spaces, the kernel provides a range of Inter-Process Communication (IPC) mechanisms so that cooperating processes can exchange data and synchronize their activities [13]. The oldest and cheapest of these mechanisms are signals, which notify a process asynchronously of an event, such as a keyboard interrupt or an illegal memory access, but which carry no data payload of their own [13].

Pipes provide a simple, one-directional byte stream between two related processes, typically a parent and its child, while named pipes (FIFOs) extend this idea to unrelated processes by exposing a persistent name in the file system [14], [15]. For more structured communication, Linux implements message queues in two variants: the older System V API, in which each queue is identified by a system-wide numeric key and messages carry an application-defined type field that a reader can filter on, and the newer POSIX API, available since kernel 2.6.6, which offers priority-ordered delivery and a file-descriptor-like interface that integrates naturally with functions such as poll() and select() [15].

Shared memory is generally the fastest IPC mechanism available, since it lets two or more processes map the same physical memory pages into their own address spaces and exchange data without any copying through the kernel; because the kernel imposes no access ordering on this shared region, applications must coordinate access explicitly, typically using semaphores [15], [13]. Semaphores themselves are simple non-negative integer counters that support only two atomic operations, conventionally called wait and signal, and are used throughout the kernel and by applications to enforce mutual exclusion over shared resources [15]. Finally, Unix domain sockets offer a flexible, bidirectional, stream- or datagram-oriented channel between processes on the same host, sharing the same programming interface as network sockets but avoiding network-stack overhead entirely [14], [16].

Table 1: Comparison of System V IPC and POSIX IPC

Aspect	System V IPC	POSIX IPC
Identification	Integer key, typically generated with ftok()	Name string, similar to a file path
Interface style	Dedicated system calls (e.g., shmget, msgget)	File-descriptor-like handles (e.g., shm_open, mq_open)
Thread-safety	Not guaranteed by design	Designed to be thread- and process-safe
Event-loop integration	Limited	Integrates with poll()/select()/epoll()
Introduced in Linux	Present since early Linux releases	Message queues since kernel 2.6.6

6. Memory Management Approaches

Like most modern operating systems, Linux gives each process the illusion of a large, contiguous, private address space through virtual memory, with per-process page tables mapping virtual addresses onto physical page frames [17]. This indirection allows the kernel to isolate processes from one another, to over-commit memory beyond what is physically installed, and to relocate or swap out pages transparently to user-space programs [17], [18]. Linux populates most pages lazily through demand paging, and uses copy-on-write when a process is duplicated by fork(), so that parent and child continue to share physical pages until one of them writes to a shared page, at which point a private copy is created [18].

To decide which pages to reclaim under memory pressure, the kernel maintains active and inactive page lists and periodically ages pages based on how recently and how often they have been referenced; pages that are rarely used are the first candidates to be written out to swap space when physical memory runs low [18].

Kernel Memory Allocators

Below this process-level view, the kernel itself needs to allocate and free memory of various sizes for its own internal data structures. The buddy system is the foundational allocator for physical page frames: memory is tracked in power-of-two-sized blocks called orders, from a single 4 KB page (order 0) up to, on typical x86 systems, a 4 MB block (order 10), and an allocation request is satisfied by the smallest available block large enough to hold it, splitting larger free blocks as needed and merging (“buddying”) adjacent free blocks back together when memory is released [19]. This scheme minimizes external fragmentation but can still waste memory internally: a 33-page request, for example, must be rounded up to a 64-page block, wasting 31 pages [19].

The slab allocator (and its modern successor, SLUB) sits above the buddy allocator and addresses this internal-fragmentation problem for small, frequently allocated kernel objects, such as inode or dentry structures. It carves the pages it receives from the buddy allocator into fixed-size chunks and maintains per-object-type caches of ready-to-use chunks, so that common allocations avoid touching the buddy allocator on every call [19]. Administrators can inspect the state of both allocators at runtime through the /proc/buddyinfo and /proc/slabinfo pseudo-files [19].

7. File System Management Approaches

Linux is able to support a large number of different, and mutually incompatible, on-disk and network file system formats, such as ext4, XFS, Btrfs, and NFS, through an abstraction layer known as the Virtual File System (VFS) [20]. The VFS defines a common object model, comprising the superblock, inode, dentry, and file objects, together with associated operation tables, that every concrete file system driver must implement; user-space programs then interact with any mounted file system through the same familiar system calls, such as open(), read(), and write(), without needing to know which underlying file system is actually in use [20], [21].

The superblock object holds metadata that describes an entire mounted file system instance, such as its block size and the location of its inode table, and on-disk file systems typically store a persistent copy of this information at a fixed location on the partition [21], [20]. The inode object represents an individual file or directory, storing attributes such as ownership, permissions, size, and timestamps, together with the information needed to locate the file's actual data blocks; a single inode may be referenced by more than one directory entry through hard links [21], [20]. Because resolving a textual path such as /home/user/file.txt into an inode would otherwise require repeated disk access, the VFS maintains a dentry (directory entry) cache that maps recently used path components onto their corresponding inodes, dramatically accelerating repeated file system lookups [20], [21]. Finally, the file object records the state of an individual open file description, such as the current read/write offset, and exists only in memory for the lifetime of the open file [20].

As the default file system on most Linux distributions, ext4 exemplifies these VFS abstractions concretely: it is a journaling file system, meaning that metadata changes (and optionally data changes) are first written to an on-disk journal before being applied to their final location, allowing the file system to recover to a consistent state quickly after an unclean shutdown rather than requiring a lengthy full-disk consistency check [21].

8. Device Management Approaches

Following the tradition established by Unix, Linux organizes peripheral hardware into three broad categories of devices: character devices, which transfer data as an unstructured stream and do not support random access, such as keyboards and serial ports; block devices, such as disks, which support random access to fixed-size blocks of data and are usually associated with a file system; and network devices, which exchange discrete packets asynchronously rather than a continuous stream for a single client [22].

Each character or block device is represented in user space by a special device file, conventionally located under /dev, and identified internally by a major number, which selects the driver responsible for the device, and a minor number, which distinguishes between multiple devices handled by the same driver [22].

sysfs and udev

Modern Linux systems manage the population of /dev dynamically rather than through a static, pre-populated directory. The kernel exposes its internal device model to user space through sysfs, a pseudo-file system, available since the 2.6 kernel series, that presents devices, drivers, and their attributes as a browsable hierarchy of virtual files under /sys [23], [24]. Whenever a device is added to or removed from the system, for example, when a USB drive is plugged in, the kernel emits a “uevent” that is picked up in user space by udev, the standard Linux device manager, which consults its configured rules to decide what device node to create under /dev, what permissions to assign it, and what symbolic links or auxiliary programs to run in response [22], [24].

This division of labour, with the kernel maintaining an authoritative, in-memory device model exposed through sysfs, and udev translating that model into concrete, correctly named and permissioned device nodes, is what allows contemporary Linux distributions to support hotplugging of USB, storage, and network hardware without requiring the administrator to pre-configure every possible device in advance [22], [23]. Device drivers themselves are very often implemented as loadable kernel modules, discussed in Chapter 3, that are loaded automatically by the module loader as soon as udev or the kernel recognizes a matching piece of hardware [7], [22].

9. Summary

This case study has traced the Linux operating system from its origin as a 1991 hobby project by a Finnish computer science student to its current position as one of the most widely deployed kernels in computing, spanning servers, cloud infrastructure, mobile devices, embedded systems, and supercomputers [5]. Architecturally, Linux combines a monolithic kernel's performance with a substantial degree of runtime flexibility through its loadable kernel module mechanism, which allows device drivers, file systems, and other subsystems to be inserted into, or removed from, a running kernel on demand [7].

In process management, the kernel's approach to CPU scheduling has itself evolved considerably, moving from the O(1) scheduler, through the long-serving Completely Fair Scheduler, to the more recent EEVDF scheduler adopted from kernel 6.6 onward, in a continuing pursuit of fairness with more predictable latency [9], [11]. A rich collection of inter-process communication primitives, from simple signals and pipes to shared memory, message queues, semaphores, and sockets, gives application developers the tools needed to coordinate cooperating processes for a wide variety of use cases [13], [15].

Underneath these user-visible facilities, Linux's memory management subsystem layers a slab allocator over a buddy-system page allocator to serve the kernel's own internal allocations efficiently, while a virtual-memory subsystem based on demand paging and copy-on-write gives every process an isolated, oversubscribable address space [18], [19]. File system access is unified across many incompatible on-disk formats by the Virtual File System abstraction, and device management has moved from static device files toward a dynamic model built on the sysfs pseudo-file system and the udev device manager, allowing modern distributions to support hotpluggable hardware with minimal manual configuration [20], [24]. Taken together, these subsystems illustrate how sustained, open, collaborative development has allowed Linux to grow, over more than three decades, from a personal hobby project into critical infrastructure for the modern computing landscape [4].

References

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ, USA: Wiley, 2018, ch. 20, pp. 713-850.

[2] A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed. Boston, MA, USA: Pearson, 2015, ch. 10, pp. 775-819.

[3] Wikipedia, "History of Linux." [Online]. Available: https://en.wikipedia.org/wiki/History_of_Linux

[4] Wikipedia, "Linux kernel." [Online]. Available: https://en.wikipedia.org/wiki/Linux_kernel

[5] TuxCare, "Linux Evolution: A Comprehensive Timeline." [Online]. Available: https://tuxcare.com/blog/linux-evolution/

[6] endoflife.date, "Linux Kernel." [Online]. Available: https://endoflife.date/linux

[7] GeeksforGeeks, "Linux Loadable Kernel Module (LKM)." [Online]. Available: https://www.geeksforgeeks.org/linux-unix/linux-loadable-kernel-module/

[8] Linux Kernel Labs, "Kernel modules." [Online]. Available: https://linux-kernel-labs.github.io/refs/heads/master/labs/kernel_modules.html

[9] Wikipedia, "Completely Fair Scheduler." [Online]. Available: https://en.wikipedia.org/wiki/Completely_Fair_Scheduler

[10] The Linux Kernel documentation, "CFS Scheduler." [Online]. Available: https://docs.kernel.org/scheduler/sched-design-CFS.html

[11] The Linux Kernel documentation, "EEVDF Scheduler." [Online]. Available: https://docs.kernel.org/scheduler/sched-eevdf.html

[12] J. Corbet, "An EEVDF CPU scheduler for Linux," LWN.net, Mar. 9, 2023. [Online]. Available: https://lwn.net/Articles/925371/

[13] EmbLogic, "Introduction to IPC on Linux." [Online]. Available: https://www.emblogic.com/blog/07/introduction-to-ipc-on-linux/

[14] M. Sharma, "Understanding Interprocess Communication (IPC): Pipes, Message Queues, Shared Memory, RPC, Semaphores, Sockets," Medium, Dec. 2023. [Online]. Available: https://mohitdtumce.medium.com/understanding-interprocess-communication-ipc-pipes-message-queues-shared-memory-rpc-902f918fba58

[15] Scaler Topics, "What Is Inter-Process Communication In Linux?" [Online]. Available: https://www.scaler.com/topics/ipc-in-linux/

[16] linuxvox.com, "Linux Interprocess Communication: An In-Depth Guide." [Online]. Available: https://linuxvox.com/blog/linux-interprocess-communication/

[17] Linux Journal, "Linux Memory Management: Understanding Page Tables, Swapping, and Memory Allocation." [Online]. Available: https://www.linuxjournal.com/content/linux-memory-management-understanding-page-tables-swapping-and-memory-allocation

[18] Enterprise Networking Planet, "Linux Virtual Memory: Optimizing Virtual Memory on Linux." [Online]. Available: https://www.enterprisenetworkingplanet.com/management/understand-linux-virtual-memory-management/

[19] University of Warsaw (MIM), "Memory management: Zones, Buddy allocator, Slab allocator" (lecture notes). [Online]. Available: https://students.mimuw.edu.pl/ZSO/Wyklady/06_memory2/BuddySlabAllocator.pdf

[20] The Linux Kernel documentation, "Overview of the Linux Virtual File System." [Online]. Available: https://docs.kernel.org/filesystems/vfs.html

[21] Its Linux FOSS, "What is a Superblock, Dentry, Inode and a File?" [Online]. Available: https://itslinuxfoss.com/what-is-superblock-inode-dentry-and-file/

[22] linuxvox.com, "Linux Device Management: A Comprehensive Guide." [Online]. Available: https://linuxvox.com/blog/linux-device-management/

[23] A. Sanjay, "Understanding the Interaction: sysfs, udev, and Character Device Drivers," Medium, Sep. 2023. [Online]. Available: https://medium.com/@anamika.sanjay16/understanding-the-interaction-sysfs-udev-and-character-device-drivers-7edb0845f916

[24] Linux.com, "Udev: Introduction to Device Management In Modern Linux System." [Online]. Available: https://www.linux.com/news/udev-introduction-device-management-modern-linux-system/